

Persistence (II)

File System - 1

中国人民大学信息学院

卞昊穹

bianhq@ruc.edu.cn

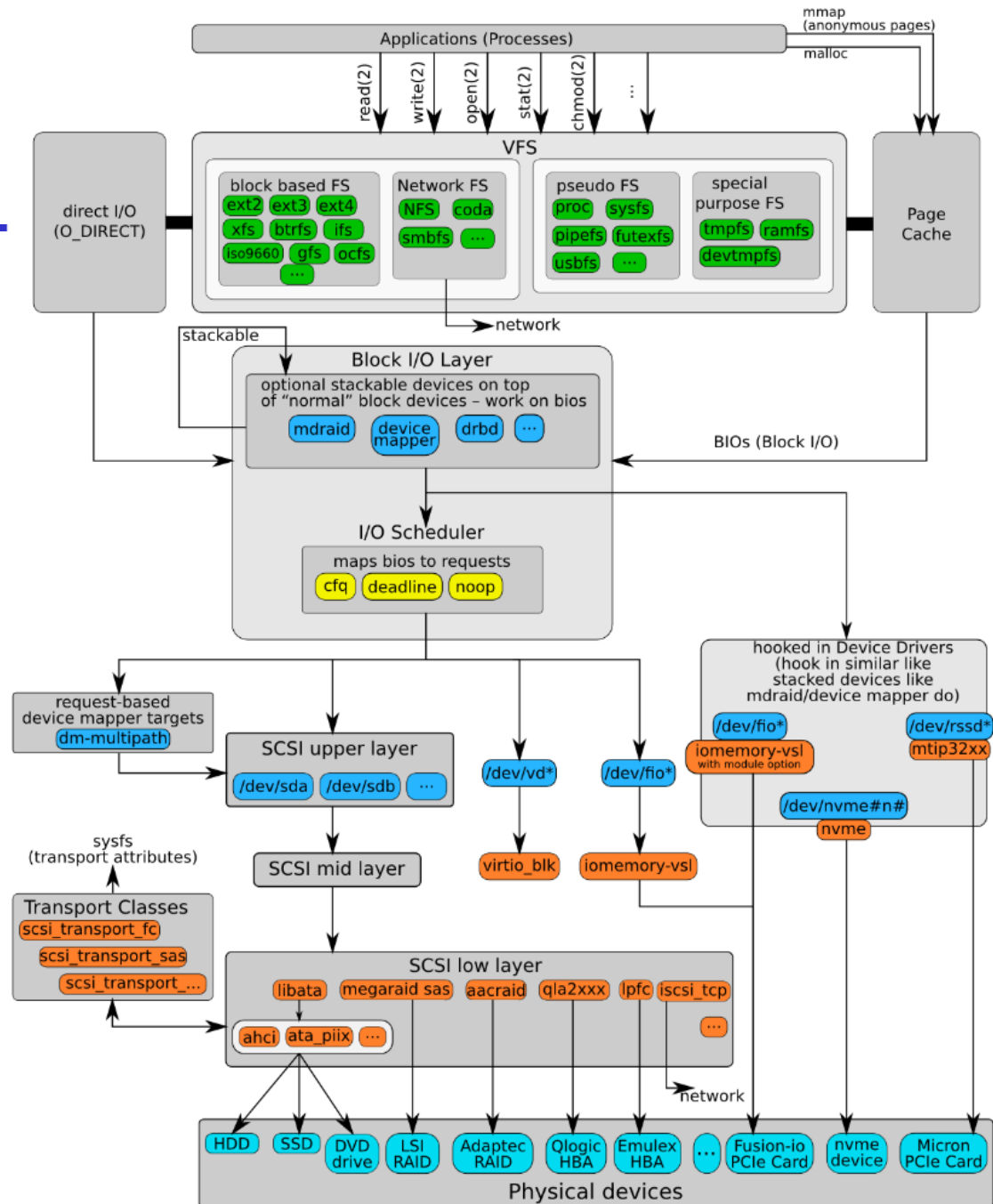
内部资料，请勿外传

Outline

- Linux I/O Stack
- File System
- File System Implementation

Linux I/O Stack

- VFS
- FS
 - (Page Cache)
- Block I/O Layer
 - I/O Scheduler
- Block Device Driver



Linux I/O Stack

- VFS (Virtual File System)
 - 很薄的一层
 - 对上提供各种文件操作接口
 - 屏蔽不同文件系统的细节，提供统一接口（**Unix IO**）
 - Open
 - Read
 - Write
 - Close
 - Stat
 - Chmod
 - ...

Linux I/O Stack

- File System

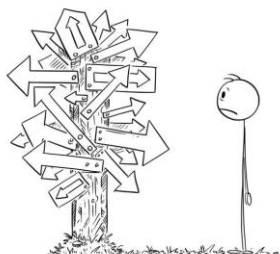
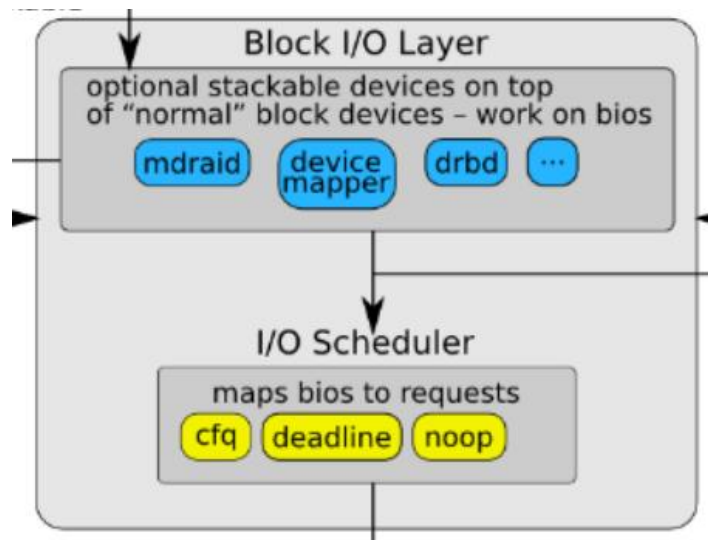
- 具体文件系统，有很多经典实现
- Ext系列、xfs、btrfs
- 网络文件系统（nfs, ceph）
- Special purpose fs (tmpfs, ramfs)
- Pseudo fs (proc, pipefs)

Linux I/O Stack

- Page Cache
 - 充分利用内存空闲空间，为慢速I/O设备提供写缓冲（buffering）和读缓存（caching）
 - Dirty数据定期写回磁盘
 - 清空page cache方法：
 - To free pagecache, use echo 1 > /proc/sys/vm/drop_caches;
 - To free dentries and inodes, use echo 2 > /proc/sys/vm/drop_caches;
 - to free pagecache, dentries and inodes, use echo 3 > /proc/sys/vm/drop_caches.
 - Ps. Storage Device和RAID卡上可能还有cache

Linux I/O Stack

- Block I/O Layer (General Block Layer)
 - 为FS提供抽象的“block device”
 - 实现“software RAID”，即multiple-device (md)
 - 对发往物理设备的I/O请求进行调度



各种block、page之类的，很混乱。。。

page, block, logical, physical...

- These words are somehow abused in CS
- 不同上下文中的含义不同
 - SSD中的page、physical block、logical block，和OS中的page和block，是不同的概念，他们之间存在一定的对应关系
 - SSD FTL中的logical block在OS中通常叫physical sector或者physical block

sector不是磁盘内部的**physical unit**吗？怎么**OS**里也有**sector**。。。

两种addressing method的纠葛: CHS vs. LBA

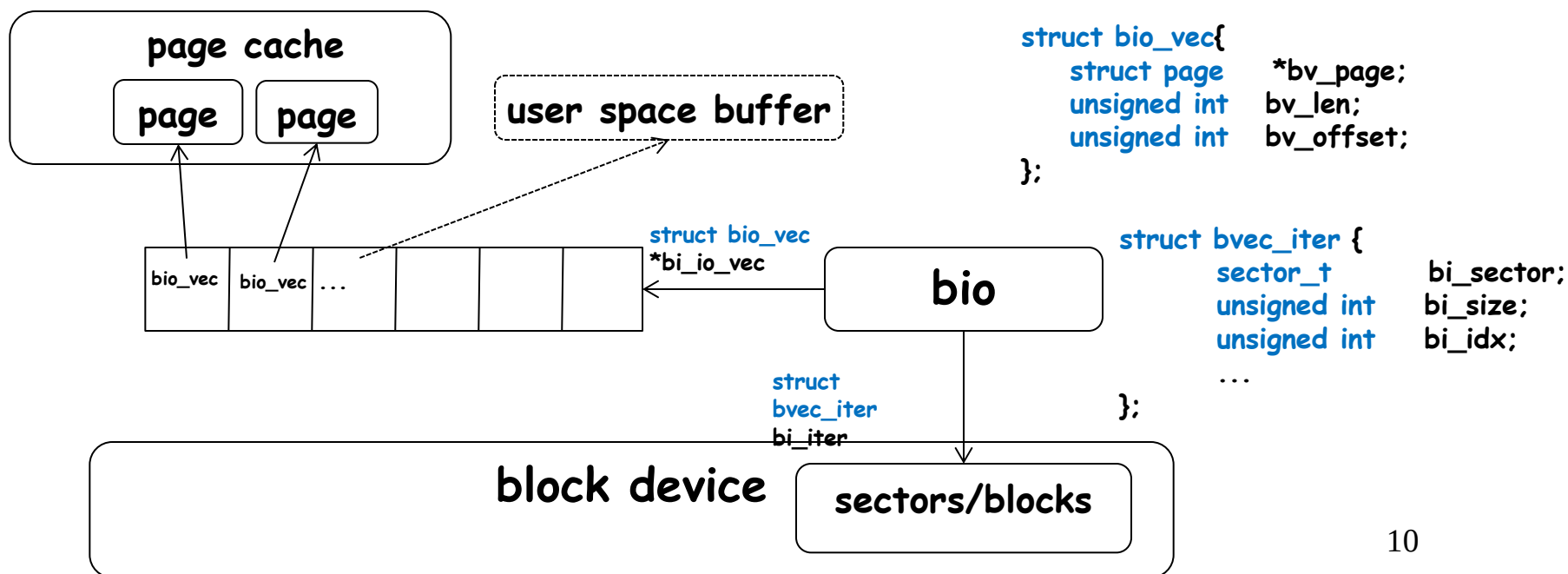
- CHS: cylinder-head-sector
 - 早期disk上没有controller，只能把CHS的三维结构暴露给OS，OS中只能使用CHS addressing来访问device
 - late 1980s: disk越来越复杂，开始加入controller以隐藏内部的几何结构，只向OS暴露logical的CHS地址
- LBA: logical block addressing
 - mid 1990s: 更加简洁的LBA开始逐渐取代CHS
 - 但直到今天，很多OS和磁盘管理工具中还有CHS的身影，一些device依然支持CHS addressing，很多资料里还在用sector作为OS访问device的基本单位

Linux中为了兼容老设备，依然有512B的sector，但对较新的设备会使用4KB block/sector (advanced format)

Linux I/O Stack

- Block I/O Layer

- struct bio代表FS对block I/O layer的I/O请求
- bio就像一个快递员，从内存中收取多个包裹，送往block device上的一个目的地，或者反之



Linux I/O Stack

- Block I/O Layer
 - Linux Device Mapper机制在block I/O layer
 - 可以将多个physical device封装为一个block device
 - 支持software RAID (i.e., multiple device,缩写md)
 - 或者自定义功能
 - 例如实现SSD为HDD提供cache功能 (Flashcache, bcache, dm-cache)
 - 实现几个kernel接口, 编译为xxx.ko, insmod xxx.ko
 - ko = kernel object
 - insmod或者modprobe命令可以用来加载模块到kernel

Linux I/O Stack

- I/O Scheduler

- 对发往device的request进行排序和调度
- 常见的调度算法
 - **noop**: no operation, 没有排序, 最简单 (适合低层是SSD)
 - **deadline**: 保证request在一定时间内被发往device
 - 四个队列, 其中两个分别为正常 **read** 和 **write** 队列, 另外两个为超时 **read** 和 **write** 的队列
 - 正常队列: 按**sector addr**排序, 合并相邻的请求; 如果新请求总在相邻地址, 那么其他地址的 **io** 请求可能饿死
 - 超时 **read** 和 **write** 的队列: 按请求创建时间排序, 保证超时的请求会优先被处理, 防止请求被饿死

Linux I/O Stack

- I/O Scheduler
 - 对发往device的request进行排序和调度
 - 常见的调度算法
 - CFQ (completely fair queuing)
 - 试图给所有进程提供完全公平的 IO 调度
 - 它为每个进程创建一个同步 IO 队列，OS以时间片(time slice)轮转的方式来执行每个进程队列中的请求，以此保证每个进程的 IO 资源占用是公平的

Linux I/O Stack

- I/O Scheduler

- 新型的SSD，内部硬件特征复杂，与磁盘有很大区别
 - 例如多通道，FTL算法
- 对于一些新设备，OS并不能给出很好的调度（厂家也不愿意公开某些细节的关乎性能的设计）
- 所以，对于某些新存储硬件，可以by-pass I/O scheduler，直接由应用程序配合设备上的控制器来完成I/O调度

Outline

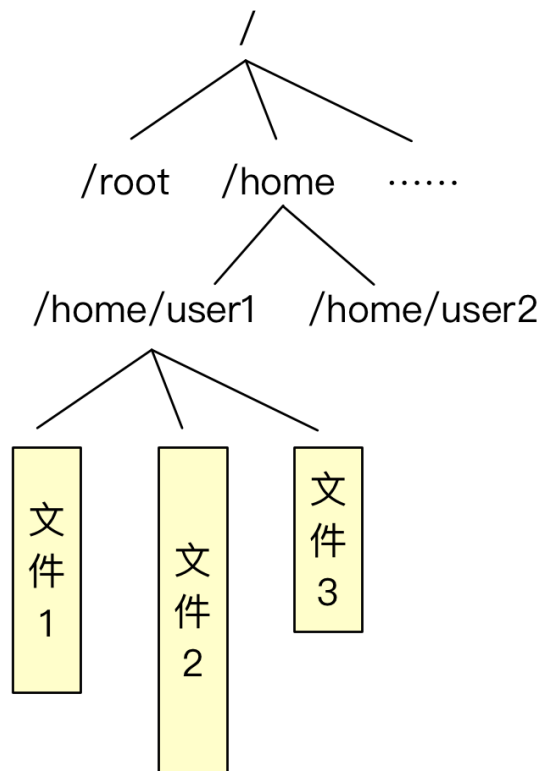
- Linux I/O Stack
- **File System**
- File System Implementation

File System

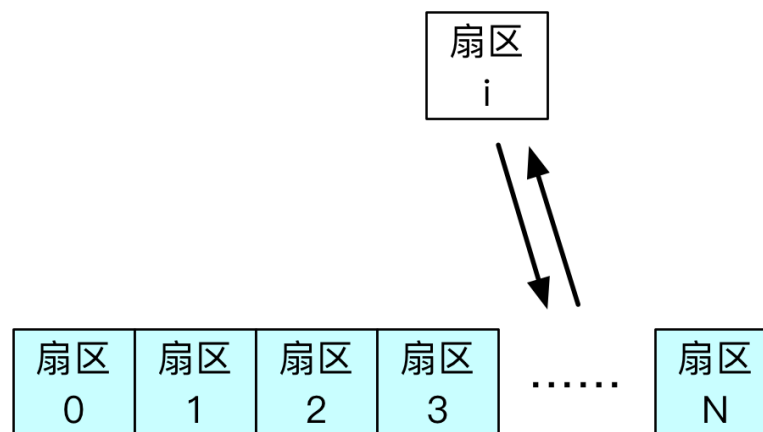
- 虚拟内存和**Block I/O**: 硬件抽象
- 文件: 数据抽象
 - 文件是一个字节向量, 不用关心其在设备上的存储细节
 - **Naming**, 文件之间的命名和关系, 重启后还能找到
 - 目录树
- 更高级的抽象
 - Key-Value store
 - object store
 - Relation (Table)
 - Graph
 - ...

File System

- 文件数据模型比**block**模型、磁盘模型更便于用户使用（更高级/复杂）



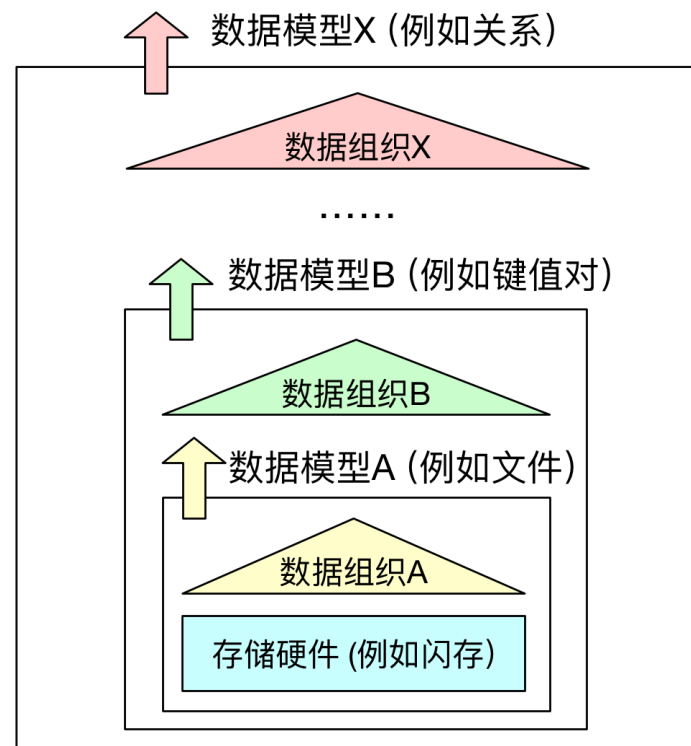
(A) 文件数据模型



(B) 磁盘的访问模型

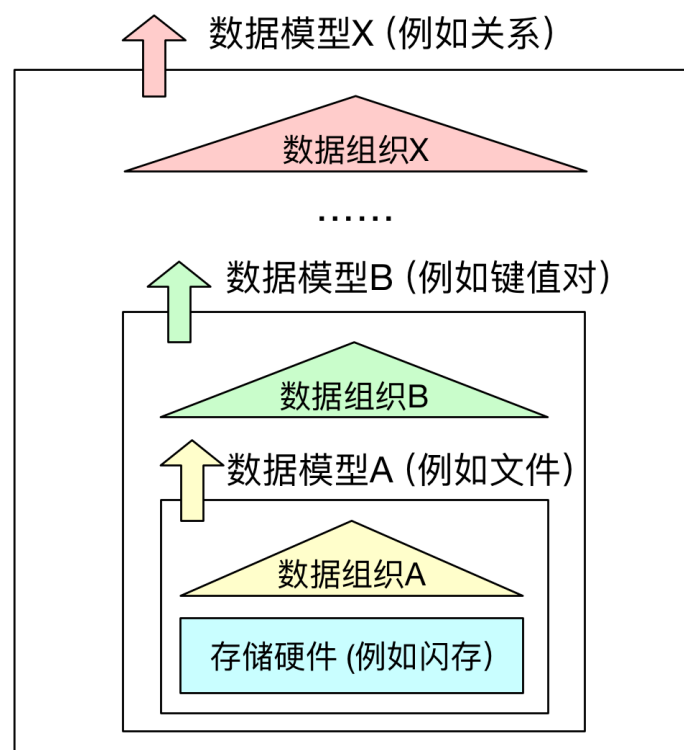
File System

- 数据模型之间的嵌套关系
 - 分布式文件系统**HDFS**系统是在每个结点的本地文件系统的基础上，将分布式的资源进行封装，对上提供文件数据模型的接口
 - **HBase**系统就是在**HDFS**的基础上，进一步封装，提供键值访问和列存数据模型的接口



File System

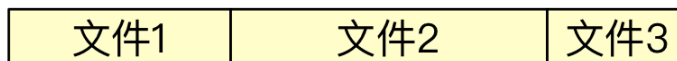
- 数据模型之间的嵌套关系
 - 例如**RocksDB**键值存储引擎就是在本地文件系统（例如**ext4**）的基础上通过日志合并（**LSM**）树结构实现了键值数据组织，提供键值数据模型的访问接口。
 - 分布式系统**Ceph**首先实现一套分布式的对象存储（键值数据模型），然后在对象存储的基础上，又进一步封装为块数据模型和文件数据模型的访问接口（通过额外实现文件系统的目录结构等功能）。



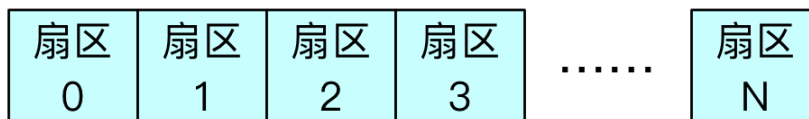
File System

- 如何实现文件系统？
 - 最直接的想法是将所有文件首尾拼接，从目录树的复杂结构退化为一维线性结构，以便和磁盘的一维**sector/block**模型相匹配
 - 但是这样做存在一些比较严重问题：
 - 文件大小增加如何处理？需要在其他地方放置文件新增的数据，那么就破坏了文件首尾相接的结构；
 - 文件删除如何处理？那么文件首尾相接的结构中会出现一些空洞，想要回收这些空间要额外的设计；
 - 如何能够快速找到用户想要访问的某个文件？按照目前的方法，可能需要遍历整个磁盘空间才找到目标文件，性能非常差。
 - 在哪里记录文件的属性信息？包括文件的访问权限、修改时间等等。

逻辑数据：



物理数据：



Outline

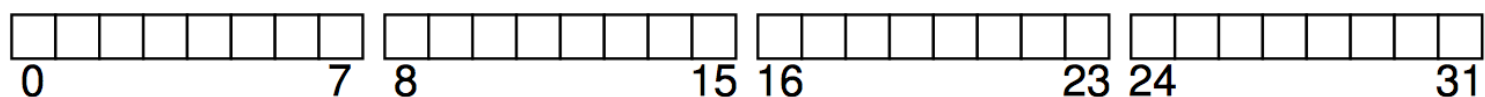
- Linux I/O Stack
- File System
- **File System Implementation**

File System Implementation

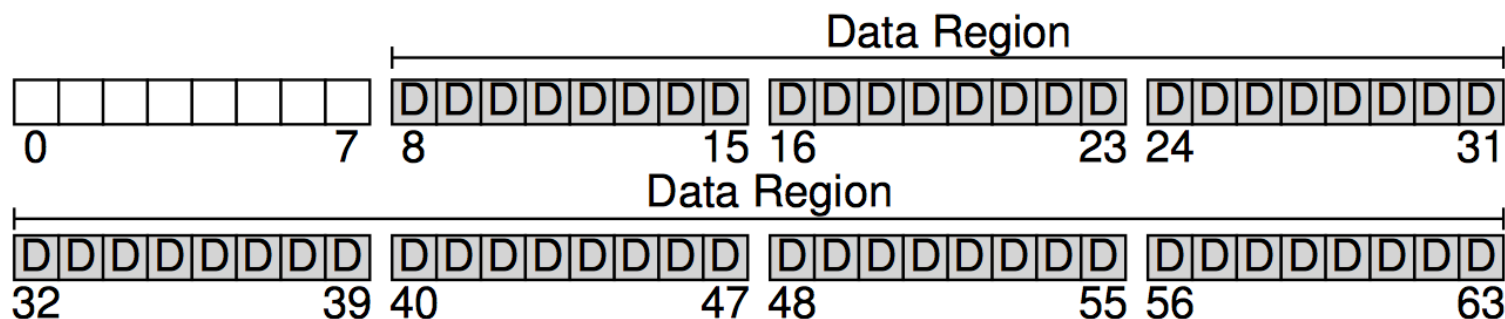
- FS本质
 - 硬件提供的接口非常简单
 - CHS或LBA地址
 - sector或block访问粒度（例如512B或4KB）
 - FS是在简单硬件接口上提供文件数据模型
 - 目录树、文件元数据、空间管理
 - KV、数据库、Graph也是如此，只是结构比FS更复杂，可能基于FS来做

File System Implementation

- VSFS (Very Simple File System)
 - 假设硬件只有64个block，每个block 4KB
 - 暂时不考虑针对硬件特性的优化

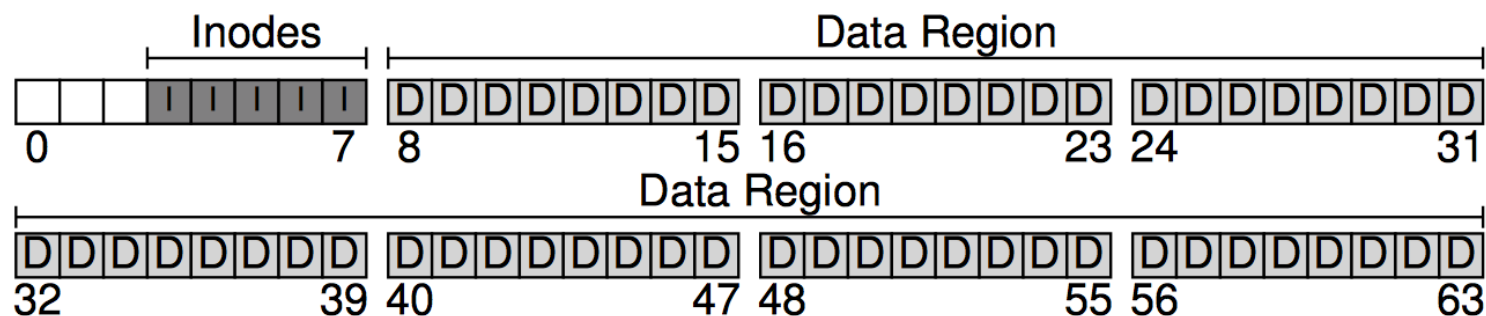


- 后面56个块作为data region



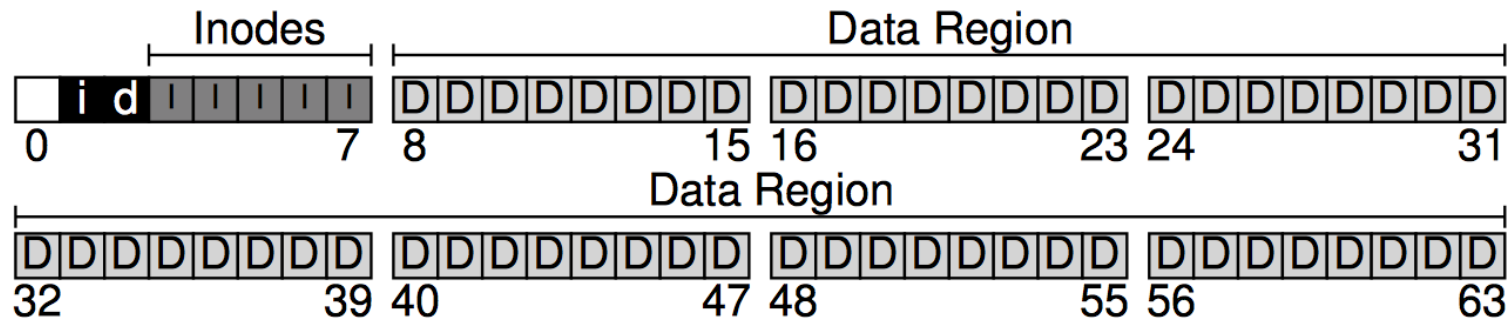
File System Implementation

- Inode
 - 文件的元数据，每个文件对应一个inode
- Inode Table
 - 比文件小很多，假设占用5个block，每个inode 256B，共80个inode，最多支持80个文件



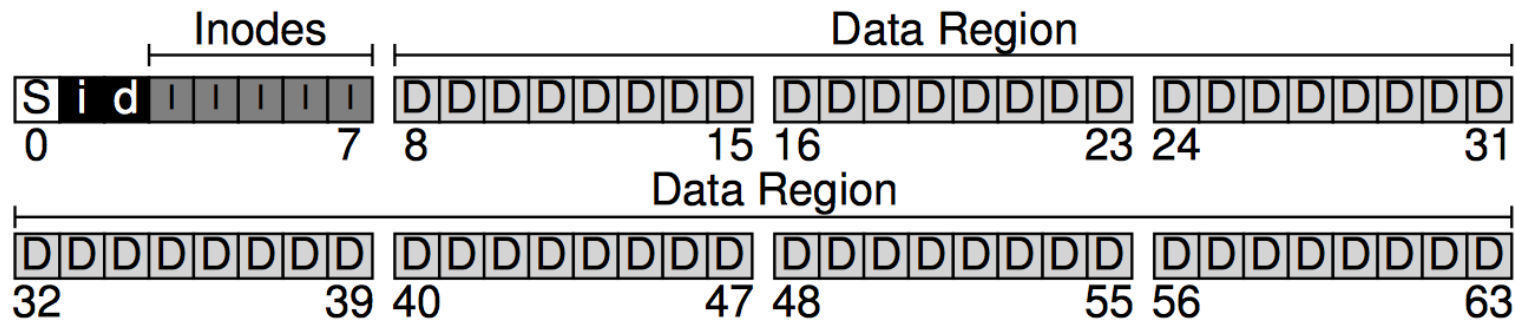
File System Implementation

- Bitmap
 - 记录inode和data block是否被占用
 - Inode bitmap (I)
 - Data bitmap (D)



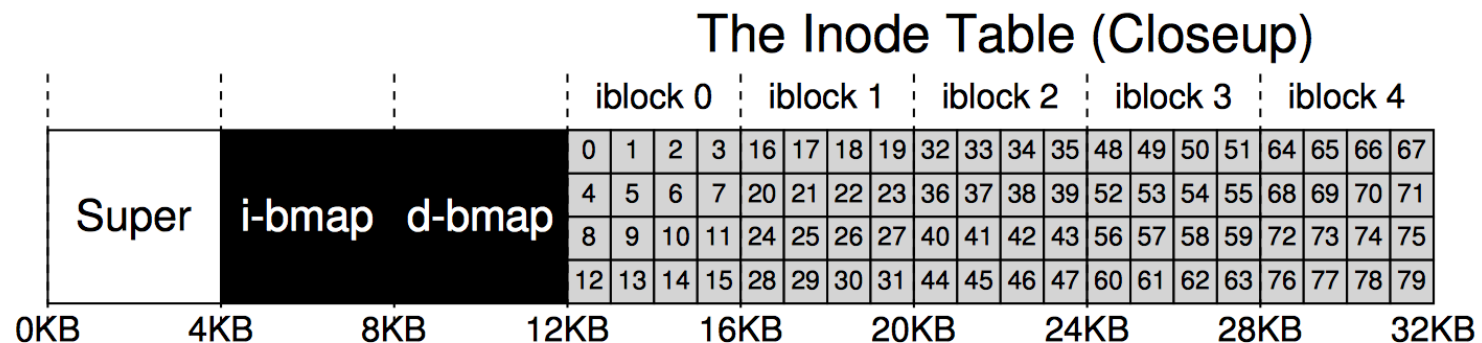
File System Implementation

- Superblock
 - 文件系统的一些参数，例如有多少inode, data block, inode table的位置等



File System Implementation

- Inode
 - i-number: 每个inode都有一个隐式的编号
 - 根据i-number能知道inode在磁盘上的准确位置



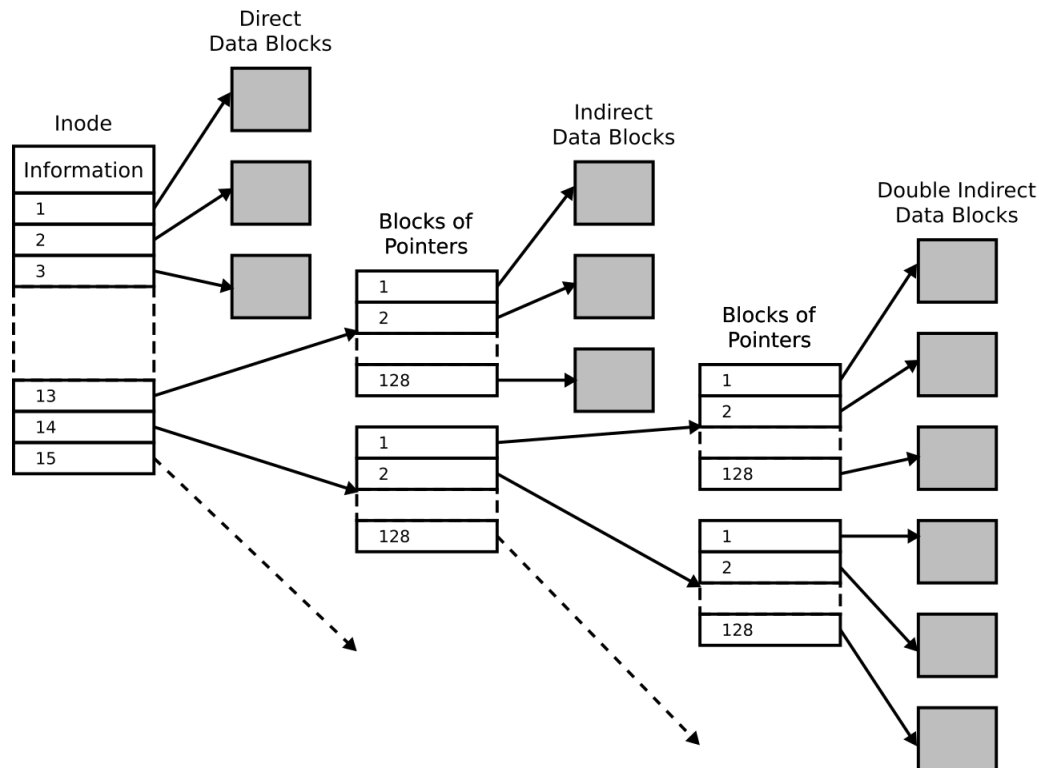
File System Implementation

- Inode数据结构举例（ext2简化）
 - 15个指针(direct pointer), 指向data blocks; 15个不够用的话, 要用到multi-level index

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists

File System Implementation

- Multi-level index
 - ext2, ext3, Netapp's WAFL



File System Implementation

- Multi-level index
 - 12 direct pointer + 1 single indirect point + 1 double indirect pointer
 - 一个文件最大的大小是多少?
 - 4GB
 - 一个block 4KB, 一个指针4B, 可放1024个指针
 - $(12 + 1024 + 1024 * 1024) * 4KB = 4GB$
 - 为什么不用平衡树?
 - Most files are small

File System Implementation

- Extent
 - Point + length (in blocks)
 - 当一个文件包含很多连续的data blocks时，节省Inode空间
 - 例如ext4, XFS
 - 此外，也能更好地保证I/O的连续性
 - 所有存储设备的顺序访问性能都好于随机访问，RAM也不例外
 - 世界上没有真正的"Random" Access Storage

File System Implementation

- Linked-based approach
 - Inode只包含一个指针，指向第一个data block，data block最后增加一个指针，指向下一个data block
 - 做random访问时性能很差
 - 把这些指针先放到内存中来加速
 - 例如FAT

File System Implementation

- 文件系统的一些特性

Most files are small

Average file size is growing

Most bytes are stored in large files

File systems contains lots of files

File systems are roughly half full

Directories are typically small

Roughly 2K is the most common size

Almost 200K is the average

A few big files use most of the space

Almost 100K on average

Even as disks grow, file systems remain ~50% full

Many have few entries; most have 20 or fewer

File System Implementation

- Directory Organization

- 目录被当做文件，有自己的inode和data block
 - Inode的类型是directory，而不是regular file
- 目录的数据内容就是所包括的文件和子目录的信息，例如

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname

File System Implementation

- Free Space Management
 - Bitmap/freelist/B-tree
 - 创建文件
 - 查找inode bitmap, 找到一个free inode
 - 分配该inode给这个文件, 并标记已使用
 - 修改bitmap
 - 分配data block也是类似
 - 优化技巧
 - 尽量提高数据连续性, 如extent

File System Implementation

- Reading a File from Disk
 - Open ("/foo/bar", O_RDONLY)
 - 从path找到bar文件的inode
 - 1) 根目录/的inode (well known, e.g., 2)
 - 2) 根目录/的data, 找到下级目录/foo的i-number
 - 3) /foo的inode
 - 4) /foo的data
 - 5) /foo/bar的inode
 - Read
 - 1) 读/foo/bar的inode
 - 2) 读/foo/bar的data
 - 3) 写/foo/bar的inode, 更新last accessed time
 - 4) 更新in-memory open file table, offset

File System Implementation

- Writing to Disk
 - 类似的过程
 - 分配inode, data blocks, 更新bitmap
 - 写一个文件至少产生5次I/O
 - Read data bitmap
 - Write data bitmap
 - Read inode
 - Write inode
 - Write data
 - 文件系统的额外开销, 尤其是小文件

File System Implementation

- Caching and Buffering
 - 没有cache, I/O太多, 尤其是目录较深
 - 管理算法, 例如LRU
 - Write buffer
 - 批处理写入, 合并一些I/O (例如同一个inode的修改)
 - 请求合并、磁盘调度, 性能更好
 - 从Linux 2.4开始, Caching和Buffering统一由Page Cache提供

课堂练习

- 假设计算机中，每个data block的大小是4KB，每个pointer 4B，inode结构中包括12个direct pointers，2个indirect pointers，2个double indirect pointer。
 - 请问该文件系统支持的一个文件最大为多大？
 - 假设一个文件是100KB，请画出这些指针的结构。